Database 1: Designing & Implementing a Database System by Nickolas Diaz

Company: AMTRAK®

Database Type: Transportation System Database using Oracle SQL

Contents

What is Amtrak..... 2

Who will be Accessing the System..... 2

Entity Relationship (ER) model..... 3

Station-Route-Schedule..... 3

Schedule-Route_Ticket-Ticket..... 4

Amtrack Database Business Rules..... 4

Amtrack Database Cardinality..... 6

SQL CREATE TABLES..... 6

SQL FUNCTIONS..... 10

SQL TRIGGERS..... 13

SQL INDEXES..... 16

SQL INSERTS..... 16

SQL UPDATES..... 18

SQL QUERIES..... 19

SQL Constraints..... 22

References..... 23

What is Amtrak

Amtrak is the United States' own national Railroad Passenger Corporation, providing a service of transportation for passengers. Founded in 1970 by Congress, it operates a network of over 24,000 miles of route in 46 states with over 500 stations. It's a federally chartered corporation, meaning it's owned by the government. but it is structured and managed like a for-profit company. The company served 22.9 million passengers in fiscal year 2021, generating $2.1 billion in revenue and employing more than 17,100 people.

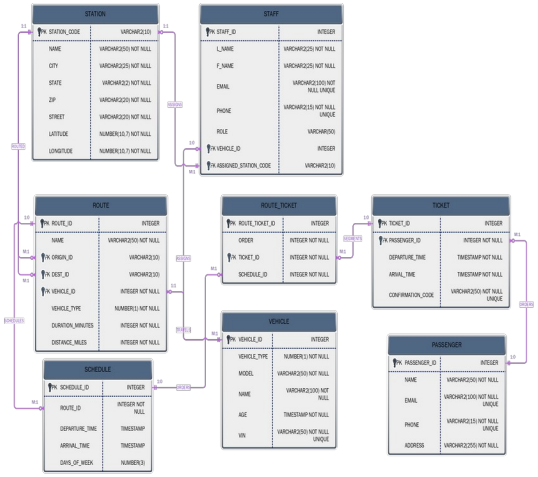
Who will be Accessing the System

Passengers: for viewing routes, schedules, tickets, and stations

Conductors/Drivers: for operating their vehicles and knowing their routes/schedules/stations

Employees: having their contact information role and assigned station

Entity Relationship (ER) model



Station-Route-Schedule

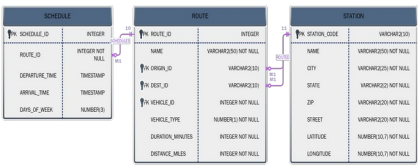
Station e.g., Tampa Union Station

Route e.g., Tampa-Orlando

Schedule e.g., Tampa-Orlando every day at 8 am for the month of April

Station one to many routes

Route one to many schedules

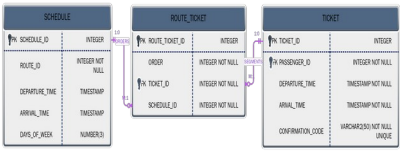


Schedule-Route_Ticket-Ticket

Ticket e.g., one origin and one destination Tampa to Miami

Route_Ticket is meant to be a junction between the many to many tables

Route_Ticket e.g., order 1 Tampa to Orlando, order 2 Orlando to Miami



Amtrack Database Business Rules

Station Rules

1. Each station must have a unique station code within 10 characters.
2. Station latitude/longitude must be valid within appropriate ranges.
3. Station Street, City, State, Zip must be valid formats
4. State must be two characters

Passenger Rules

1. All passengers must have a valid address, email and phone.
2. Passenger email addresses and phone numbers must be unique.

Ticket Rules

1. Tickets must be associated with a valid passenger.
2. Arrival time must be later than departure time.
3. Each ticket must have a unique confirmation code.
4. A ticket can be used for multiple routes through the ROUTE_TICKET table.

Staff Rules

1. Staff roles should be restricted to predefined categories such as (CONDUCTOR, DRIVER, CLERK).
2. All staff members must be assigned to a station.
3. All Conductors must be assigned to a Train and Drivers assigned to a Bus.
4. Staff email addresses and phone numbers must be unique across the system and consist of their last name + first name + staff id + @amtrak.com.

Vehicle Rules

1. Each vehicle must have a unique VIN.
2. Vehicle type must be either train (1) or bus (0).
3. The vehicle age is a timestamp of when it was acquired.

Route Rules

1. Origin and destination stations must be different.
2. Route duration and distance must be positive values.
3. Route duration is the typical amount of time to get to the destination.
4. Routes must be assigned to a vehicle of the appropriate type.

Schedule Rules

1. Schedules must be associated with a valid route.
2. Departure time must be earlier than arrival time.
3. Days of week must use a valid bitmask value from 0 to 127.
4. Recurring schedules should not conflict with the same vehicle.

Route_Ticket Rules

1. Each route ticket must link a valid ticket to a valid schedule.
2. A passenger cannot be booked on overlapping schedules.

3. Order is used to track the order of routes needed to get to one station to another

Amtrack Database Cardinality

FOREIGN KEY(COLUMN)	FOREIGN KY -> PRIMARY KY	CARDINALITY
TICKET(PASSENGER_ID)	TICKET -> PASSENGER	M:1,1:0
STAFF(VEHICLE_ID)	STAFF -> VEHICLE	M:1,1:1
STAFF(ASSIGNED_STATION_CODE)	STAFF -> STATION	M:1,1:1
ROUTE(VEHICLE_ID)	ROUTE -> VEHICLE	M:1,1:0
ROUTE(ORIGIN_ID)	ROUTE -> STATION	M:1,1:1
ROUTE(DEST_ID)	ROUTE -> STATION	M:1,1:1
SCHEDULE(ROUTE_ID)	SCHEDULE -> ROUTE	M:1,1:1
ROUTE_TICKET(TICKET_ID)	ROUTE_TICKET -> TICKET	M:1,1:0
ROUTE_TICKET(SCHEDULE_ID)	ROUTE_TICKET -> SCHEDULE	M:1,1:0

SQL CREATE TABLES

```sql
CREATE TABLE station
 (
   station_code VARCHAR2(10) PRIMARY KEY,
   NAME         VARCHAR2(50) NOT NULL,
   city         VARCHAR2(25) NOT NULL,
   state        VARCHAR2(2) NOT NULL,
   zip          VARCHAR2(20) NOT NULL,
   street       VARCHAR2(20) NOT NULL,
   latitude     NUMBER(10, 7) NOT NULL,
   longitude    NUMBER(10, 7) NOT NULL
 );
```

STATION	
PK STATION_CODE	VARCHAR2(10)
NAME	VARCHAR2(50) NOT NULL
CITY	VARCHAR2(25) NOT NULL
STATE	VARCHAR2(2) NOT NULL
ZIP	VARCHAR2(20) NOT NULL
STREET	VARCHAR2(20) NOT NULL
LATITUDE	NUMBER(10,7) NOT NULL
LONGITUDE	NUMBER(10,7) NOT NULL

```
CREATE TABLE passenger
 (
   passenger_id INTEGER PRIMARY KEY,
   NAME        VARCHAR2(50) NOT NULL,
   email       VARCHAR2(100) NOT NULL UNIQU
E,
   phone       VARCHAR2(15) NOT NULL UNIQU
E,
   address     VARCHAR2(255) NOT NULL
 );
```

PASSENGER	
PK PASSENGER_ID	INTEGER
NAME	VARCHAR2(50) NOT NULL
EMAIL	VARCHAR2(100) NOT NULL UNIQUE
PHONE	VARCHAR2(15) NOT NULL UNIQUE
ADDRESS	VARCHAR2(255) NOT NULL

```
CREATE TABLE ticket
 (
   ticket_id        INTEGER PRIMARY KEY,
   passenger_id     INTEGER NOT NULL,
   departure_time   TIMESTAMP NOT NULL,
   arrival_time     TIMESTAMP NOT NULL,
   confirmation_code VARCHAR2(50) NOT NULL
UNIQUE,
   CONSTRAINT fk_ticket_passenger FOREIGN
KEY (passenger_id) REFERENCES
   passenger(passenger_id)
 );
```

TICKET	
PK TICKET_ID	INTEGER
FK PASSENGER_ID	INTEGER NOT NULL
DEPARTURE_TIME	TIMESTAMP NOT NULL
ARIVAL_TIME	TIMESTAMP NOT NULL
CONFIRMATION_CODE	VARCHAR2(50) NOT NULL UNIQUE

```
CREATE TABLE staff
 (
   staff_id            INTEGER PRIMARY KEY,
   l_name              VARCHAR(25) NOT NULL,
   f_name              VARCHAR(25) NOT NULL,
   email               VARCHAR(100) NOT NULL U
NIQUE,
   phone               VARCHAR(15) NOT NULL UN
IQUE,
   role                VARCHAR(50),-- CONDUCTO
R, DRIVER, CLERK
   vehicle_id          INTEGER,
   assigned_station_code VARCHAR2(10),
   CONSTRAINT fk_staff_vehicle FOREIGN KEY
(vehicle_id) REFERENCES vehicle(
   vehicle_id),
   CONSTRAINT fk_staff_station FOREIGN KEY
(assigned_station_code) REFERENCES
   station(station_code)
 );
```

STAFF	
PK STAFF_ID	INTEGER
L_NAME	VARCHAR2(25) NOT NULL
F_NAME	VARCHAR2(25) NOT NULL
EMAIL	VARCHAR2(100) NOT NULL UNIQUE
PHONE	VARCHAR2(15) NOT NULL UNIQUE
ROLE	VARCHAR(50)
FK VEHICLE_ID	INTEGER
FK ASSIGNED_STATION_CODE	VARCHAR2(10)

```
CREATE TABLE vehicle
 (
   vehicle_id    INTEGER PRIMARY KEY,
   vehicle_type NUMBER(1) NOT NULL,-- 1 FO
R TRAIN, 0 FOR BUS
   model         VARCHAR2(50) NOT NULL,
   NAME          VARCHAR2(100) NOT NULL,
   age           TIMESTAMP NOT NULL,
   vin           VARCHAR(50) NOT NULL UNIQUE
 );
```

VEHICLE	
PK VEHICLE_ID	INTEGER
VEHICLE_TYPE	NUMBER(1) NOT NULL
MODEL	VARCHAR2(50) NOT NULL
NAME	VARCHAR2(100) NOT NULL
AGE	TIMESTAMP NOT NULL
VIN	VARCHAR2(50) NOT NULL UNIQUE

```
CREATE TABLE route
 (
   route_id        INTEGER PRIMARY KEY,
   NAME            VARCHAR2(50) NOT NULL,
   origin_id       VARCHAR2(10),
   dest_id         VARCHAR2(10),
   vehicle_id      INTEGER NOT NULL,
   vehicle_type    NUMBER(1) NOT NULL,-- 1 F
OR TRAIN, 0 FOR BUS
   duration_minutes INTEGER NOT NULL,
   distance_miles   INTEGER NOT NULL,
   CONSTRAINT fk_route_vehicle FOREIGN KE
Y (vehicle_id) REFERENCES vehicle(
   vehicle_id),
   CONSTRAINT fk_route_origin FOREIGN KEY
(origin_id) REFERENCES station(
   station_code),
   CONSTRAINT fk_route_dest FOREIGN KEY (
dest_id) REFERENCES station(
   station_code)
 );
```

ROUTE	
PK ROUTE_ID	INTEGER
NAME	VARCHAR2(50) NOT NULL
FK ORIGIN_ID	VARCHAR2(10)
FK DEST_ID	VARCHAR2(10)
FK VEHICLE_ID	INTEGER NOT NULL
VEHICLE_TYPE	NUMBER(1) NOT NULL
DURATION_MINUTES	INTEGER NOT NULL
DISTANCE_MILES	INTEGER NOT NULL

```
CREATE TABLE schedule
 (
   schedule_id     INTEGER PRIMARY KEY,
   route_id        INTEGER NOT NULL,
   departure_time  TIMESTAMP,
   arrival_time    TIMESTAMP,
   days_of_week    NUMBER(3),
   -- MON|TUE|WED|THUR|FRI|SAT|SUN using
bit masks up to 128
   CONSTRAINT fk_schedule_route FOREIGN K
EY (route_id) REFERENCES route(
   route_id)
 );
```

SCHEDULE	
PK SCHEDULE_ID	INTEGER
ROUTE_ID	INTEGER NOT NULL
DEPARTURE_TIME	TIMESTAMP
ARRIVAL_TIME	TIMESTAMP
DAYS_OF_WEEK	NUMBER(3)

```
CREATE TABLE route_ticket
 (
   route_ticket_id INTEGER PRIMARY KEY,
   order_id        INTEGER NOT NULL,
   ticket_id       INTEGER NOT NULL,
   schedule_id     INTEGER NOT NULL,
   CONSTRAINT fk_route_ticket FOREIGN KEY (
ticket_id) REFERENCES ticket(
   ticket_id),
   CONSTRAINT fk_ticket_schedule FOREIGN K
EY (schedule_id) REFERENCES schedule
   (schedule_id)
```

ROUTE_TICKET	
PK ROUTE_TICKET_ID	INTEGER
ORDER	INTEGER NOT NULL
FK TICKET_ID	INTEGER NOT NULL
SCHEDULE_ID	INTEGER NOT NULL

);	

SQL FUNCTIONS

```
-- Passenger Email Checks whether is in the valid format
CREATE OR REPLACE FUNCTION is_valid_email(p_email VARCHAR2) RETURN BOOLEAN IS
BEGIN
   RETURN REGEXP_LIKE(p_email, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');
END;
/
```

```
-- Passenger Phone only 10-digit numbers
CREATE OR REPLACE FUNCTION is_valid_phone(p_phone VARCHAR2) RETURN BOOLEAN IS
BEGIN
   RETURN REGEXP_LIKE(p_phone, '^\d{10}$');
END;
/
```

```
-- State (2-letter code uppercase)
-- from https://en.wikipedia.org/wiki/List_of_U.S._state_and_territory_abbreviations
CREATE OR REPLACE FUNCTION is_valid_state(p_state VARCHAR2) RETURN BOOLEAN IS
BEGIN
   RETURN REGEXP_LIKE(p_state, '^[A-Z]{2}$') or p_state
in('AL','AK','AZ','AR','CA','CO','CT','DE','DC','FL','GA','HI','ID','IL','IN','IA','KS','KY','LA','ME','MD','MA','MI','MN','MS','MO','M
T','NE','NV','NH','NJ','NM','NY','NC','ND','OH','OK','OR','PA','RI','SC','SD','TN','TX','UT','VT','VA','WA','WV','WI','WY','AS',
'GU','MP','PR','VI','MH','FM','PW','AA','AE','AP','NB','CM','CZ','PI','TT');
END;
/
```

```
-- ZIP Code (either 5 or 5+4 format)
CREATE OR REPLACE FUNCTION is_valid_zip(p_zip VARCHAR2) RETURN BOOLEAN IS
BEGIN
   RETURN REGEXP_LIKE(p_zip, '^\d{5}(-\d{4})?$');
END;
/
```

```
-- Latitude: Valid range -90.0000000 to 90.0000000
CREATE OR REPLACE FUNCTION is_valid_latitude(p_lat NUMBER) RETURN BOOLEAN IS
BEGIN
   RETURN p_lat BETWEEN -90 AND 90;
END;
/
```

```oracle_plsql
-- Longitude: Valid range -180.0000000 to 180.0000000
CREATE OR REPLACE FUNCTION is_valid_longitude(p_long NUMBER) RETURN BOOLEAN IS
BEGIN
   RETURN p_long BETWEEN -180 AND 180;
END;
/
```

```oracle_plsql
-- Vehicle Type: Must be 0 or 1
CREATE OR REPLACE FUNCTION is_valid_vehicle_type(p_type NUMBER) RETURN BOOLEAN IS
BEGIN
   RETURN p_type IN (0, 1);
END;
/
```

```oracle_plsql
-- Staff Role
CREATE OR REPLACE FUNCTION is_valid_staff_role(p_role VARCHAR2) RETURN BOOLEAN IS
BEGIN
   RETURN UPPER(p_role) IN ('CONDUCTOR', 'DRIVER', 'CLERK');
END;
/
```

```oracle_plsql
--Into 7 numbers 0 for false to the the masked schedual of the days of the week
CREATE OR REPLACE FUNCTION GET_DAY_MASK (
   sun IN NUMBER,
   mon IN NUMBER,
   tue IN NUMBER,
   wed IN NUMBER,
   thu IN NUMBER,
   fri IN NUMBER,
   sat IN NUMBER
) RETURN NUMBER IS
   result NUMBER := 0;
BEGIN
   IF sun = 1 THEN result := result + POWER(2, 0); END IF;
   IF mon = 1 THEN result := result + POWER(2, 1); END IF;
   IF tue = 1 THEN result := result + POWER(2, 2); END IF;
   IF wed = 1 THEN result := result + POWER(2, 3); END IF;
   IF thu = 1 THEN result := result + POWER(2, 4); END IF;
   IF fri = 1 THEN result := result + POWER(2, 5); END IF;
   IF sat = 1 THEN result := result + POWER(2, 6); END IF;

   RETURN result;
END;
/
```

```oracle_plsql
--Returns bitmask of the day into a string
CREATE OR REPLACE FUNCTION GET_DAY_STRING (
   day_mask IN NUMBER
) RETURN VARCHAR2 IS
   result VARCHAR2(100) := '';
BEGIN
   IF day_mask = 0 OR day_mask > 127 THEN
      RETURN 'None';
   ELSE
      result := 'Every ';
   END IF;

   IF day_mask = 127 THEN RETURN result || 'Day';
   ELSIF day_mask = 65 THEN RETURN result || 'Weekend';
   ELSIF day_mask = 62 THEN RETURN result || 'Weekday';
   END IF;

   IF BITAND(day_mask, POWER(2, 0)) > 0 THEN result := result || 'Sun, '; END IF;
   IF BITAND(day_mask, POWER(2, 1)) > 0 THEN result := result || 'Mon, '; END IF;
   IF BITAND(day_mask, POWER(2, 2)) > 0 THEN result := result || 'Tue, '; END IF;
   IF BITAND(day_mask, POWER(2, 3)) > 0 THEN result := result || 'Wed, '; END IF;
   IF BITAND(day_mask, POWER(2, 4)) > 0 THEN result := result || 'Thu, '; END IF;
   IF BITAND(day_mask, POWER(2, 5)) > 0 THEN result := result || 'Fri, '; END IF;
   IF BITAND(day_mask, POWER(2, 6)) > 0 THEN result := result || 'Sat, '; END IF;

   -- Remove trailing comma if needed
   IF result IS NOT NULL THEN
      result := RTRIM(result, ', ');
   END IF;

   RETURN result;
END;
/
```

```oracle_plsql
-- returns Train for int 1 else Bus for 0
CREATE OR REPLACE FUNCTION VEHICLE_TYPE_CHAR (
   Vehicle IN INTEGER
) RETURN VARCHAR2 IS
BEGIN
   IF Vehicle = 1 THEN RETURN 'TRAIN';
   ELSE RETURN 'BUS';
   END IF;
END;

--Input a timestamp and returns something like April 2025, 02:30 PM
CREATE OR REPLACE FUNCTION FORMAT_TIMESTAMP(ts IN TIMESTAMP)
RETURN VARCHAR2 IS
BEGIN
  RETURN TO_CHAR(ts, 'Month YYYY, HH:MI PM ');
```

```
END;
/
```

```
--Input a timestamp and returns something like April 20 2025, 02:30 PM
CREATE OR REPLACE FUNCTION FORMAT_TIMESTAMP_DAY(ts IN TIMESTAMP)
RETURN VARCHAR2 IS
BEGIN
  RETURN TO_CHAR(ts, 'Month DD YYYY, HH:MI PM ');
END;
/
--Inputs two timestamps and returns the difference in minutes
CREATE OR REPLACE FUNCTION GET_MINUTES_BETWEEN(
  start_time IN TIMESTAMP,
  end_time   IN TIMESTAMP
) RETURN INTEGER IS
  diff INTERVAL DAY TO SECOND;
  total_minutes INTEGER;
BEGIN
  diff := end_time - start_time;

  total_minutes :=
      EXTRACT(DAY FROM diff) * 720 +
      EXTRACT(HOUR FROM diff) * 60 +
      EXTRACT(MINUTE FROM diff);

  RETURN total_minutes;
END;
/
```

SQL TRIGGERS

```
-- validate passenger
CREATE OR REPLACE TRIGGER trg_validate_passenger
BEFORE INSERT OR UPDATE ON PASSENGER
FOR EACH ROW
BEGIN
    IF NOT is_valid_email(:NEW.EMAIL) THEN
        RAISE_APPLICATION_ERROR(-10001, 'Invalid email format for Passenger.');
    END IF;

    IF NOT is_valid_phone(:NEW.PHONE) THEN
        RAISE_APPLICATION_ERROR(-10002, 'Invalid phone format for Passenger.');
    END IF;
END;
/
```

```oracle_plsql
-- validate staff
CREATE OR REPLACE TRIGGER trg_validate_staff
BEFORE INSERT OR UPDATE ON STAFF
FOR EACH ROW
BEGIN
   IF NOT is_valid_email(:NEW.EMAIL) THEN
      RAISE_APPLICATION_ERROR(-10004, 'Invalid email format for Staff.');
   END IF;

   IF :NEW.ROLE IS NOT NULL AND NOT is_valid_staff_role(:NEW.ROLE) THEN
      RAISE_APPLICATION_ERROR(-10005, 'Invalid role for Staff.');
   END IF;
END;
/
```

```oracle_plsql
-- validate station
CREATE OR REPLACE TRIGGER trg_validate_station
BEFORE INSERT OR UPDATE ON STATION
FOR EACH ROW
BEGIN
   IF NOT is_valid_state(:NEW.STATE) THEN
      RAISE_APPLICATION_ERROR(-10007, 'Invalid state code for Station.');
   END IF;

   IF NOT is_valid_zip(:NEW.ZIP) THEN
      RAISE_APPLICATION_ERROR(-10008, 'Invalid ZIP code format for Station.');
   END IF;

   IF NOT is_valid_latitude(:NEW.LATITUDE) THEN
      RAISE_APPLICATION_ERROR(-10009, 'Invalid latitude value for Station.');
   END IF;

   IF NOT is_valid_longitude(:NEW.LONGITUDE) THEN
      RAISE_APPLICATION_ERROR(-10010, 'Invalid longitude value for Station.');
   END IF;
END;
/
```

```oracle_plsql
-- validate vehicle
CREATE OR REPLACE TRIGGER trg_validate_vehicle
BEFORE INSERT OR UPDATE ON VEHICLE
FOR EACH ROW
BEGIN
   IF NOT is_valid_vehicle_type(:NEW.VEHICLE_TYPE) THEN
      RAISE_APPLICATION_ERROR(-10012, 'Invalid vehicle type (must be 0 or 1).');
   END IF;
END;
/
```

```oracle_plsql
-- validate schedule
CREATE OR REPLACE TRIGGER trg_validate_schedule
BEFORE INSERT OR UPDATE ON SCHEDULE
FOR EACH ROW
BEGIN
   IF :NEW.DEPARTURE_TIME IS NOT NULL AND :NEW.ARRIVAL_TIME IS NOT NULL THEN
      IF :NEW.ARRIVAL_TIME <= :NEW.DEPARTURE_TIME THEN
         RAISE_APPLICATION_ERROR(-10013, 'Arrival time must be after departure time.');
      END IF;
   END IF;
END;
/
```

```oracle_plsql
-- validate route
CREATE OR REPLACE TRIGGER trg_validate_route
BEFORE INSERT OR UPDATE ON ROUTE
FOR EACH ROW
BEGIN
   IF :NEW.DURATION_MINUTES <= 0 THEN
      RAISE_APPLICATION_ERROR(-10016, 'Route duration must be greater than zero.');
   END IF;

   IF :NEW.DISTANCE_MILES <= 0 THEN
      RAISE_APPLICATION_ERROR(-10017, 'Route distance must be greater than zero.');
   END IF;
END;
/
```

```oracle_plsql
-- autogenerate email for staff, email consists of firstname.lastnamestaffid@amtrak.com
CREATE OR REPLACE TRIGGER trg_generate_unique_email
BEFORE INSERT OR UPDATE OF F_NAME, L_NAME, EMAIL ON STAFF
FOR EACH ROW
DECLARE
   base_email VARCHAR2(100);
   unique_email VARCHAR2(100);
BEGIN
   IF :NEW.EMAIL IS NULL OR
      :NEW.F_NAME != :OLD.F_NAME OR
      :NEW.L_NAME != :OLD.L_NAME THEN

      base_email := LOWER(:NEW.F_NAME || '.' || :NEW.L_NAME) || :NEW.STAFF_ID ||  '@amtrak.com';
      unique_email := base_email;

      :NEW.EMAIL := unique_email;
   END IF;
```

```
END;
/
```

```
-- assigns a random confirmation code upperalphanumberic 10 digit code to confirmation_code from ticket
CREATE OR REPLACE TRIGGER TICKET_CONFIRMATION_CODE
BEFORE INSERT ON TICKET
FOR EACH ROW
BEGIN
   IF :NEW.CONFIRMATION_CODE IS NULL THEN
      :NEW.CONFIRMATION_CODE := DBMS_RANDOM.STRING('X', 10);
   END IF;
END;
/
```

SQL INDEXES

```
-- is for querying
CREATE INDEX IDX_STATION_CITY ON STATION(CITY);
CREATE INDEX IDX_STATION_STATE ON STATION(STATE);
CREATE INDEX IDX_SCHEDULE_ARRIVAL ON SCHEDULE(ARRIVAL_TIME);
CREATE INDEX IDX_SCHEDULE_DEPARTURE ON SCHEDULE(DEPARTURE_TIME);
CREATE INDEX IDX_PAYMENT_DATE ON PAYMENT(PAYMENT_DATE);
CREATE INDEX IDX_VEHICLE_TYPE ON VEHICLE(VEHICLE_TYPE);
CREATE INDEX IDX_ROUTE_ORIGIN_DEST ON ROUTE(ORIGIN_ID, DEST_ID); -- index for both origin and
destination
```

```
-- indexes for foreign keys
CREATE INDEX IDX_ROUTE_ORIGIN ON ROUTE(ORIGIN_ID);
CREATE INDEX IDX_ROUTE_DEST ON ROUTE(DEST_ID);
CREATE INDEX IDX_ROUTE_VEHICLE ON ROUTE(VEHICLE_ID);
CREATE INDEX IDX_TICKET_PASSENGER ON TICKET(PASSENGER_ID);
CREATE INDEX IDX_SCHEDULE_ROUTE ON SCHEDULE(ROUTE_ID);
CREATE INDEX IDX_STAFF_STATION ON STAFF(ASSIGNED_STATION_CODE);
CREATE INDEX IDX_ROUTE_TICKET ON ROUTE_TICKET(TICKET_ID);
```

SQL INSERTS

```
INSERT INTO PASSENGER (PASSENGER_ID, NAME,EMAIL, PHONE, ADDRESS) VALUES
(1,'Robert Waters','robertwalter91@gmail.com',4342101933,'2002 Magdalena Pass, Pacochahaven, Tennessee 98096'),
(2,'Alan Benson','alanbenson21@outlook.com',2729860902,'32885 Winona Isle, New Maureen, Arizona - 91241, Saint Martin'),
(3,'Janet Friedman','janetfriedman63@yahoo.com',6626732300,'2834 Laurianne Loaf, Ortizberg, New Mexico - 42339, Eritrea'),
(4,'Eunice Griffith','eunicegriffith78@hotmail.com',8058295046,'79241 Doug Center, Lake Mafaldaworth, Arkansas - 83793, Papua
```

```
New Guinea'),
(5,'Matt Oneal','mattoneal46@protonmail.com',8402774545,'463 Mervin Junctions, Port Keon, Colorado - 73593, Australia');
```

```
INSERT INTO STATION (STATION_CODE, NAME, CITY, STATE, ZIP, STREET, LATITUDE, LONGITUDE) VALUES
('TPA', 'Tampa Union Station', 'Tampa', 'FL', '33602', '601 N Nebraska Ave', 27.9515, -82.4552),
('ORL', 'Orlando Amtrak Station', 'Orlando', 'FL', '32806', '1400 Sligh Blvd', 28.5199, -81.3820),
('MIA', 'Miami Amtrak Station', 'Miami', 'FL', '33147', '8303 NW 37th Ave', 25.8115, -80.2227),
('WPB', 'West Palm Beach Amtrak Station', 'West Palm Beach', 'FL', '33401', '203 S Tamarind Ave',26.7118, -80.0526),
('JAX', 'Jacksonville Amtrak Station', 'Jacksonville', 'FL', '32209', '3570 Clifford Ln', 30.3392, -81.6543);
```

```
INSERT INTO TICKET (TICKET_ID, PASSENGER_ID, DEPARTURE_TIME, ARRIVAL_TIME, CONFIRMATION_CODE) VALUES
(101, 1, TO_TIMESTAMP('2025-04-15 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-15 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), null),
(102, 2, TO_TIMESTAMP('2025-04-16 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-16 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), null),
(103, 3, TO_TIMESTAMP('2025-04-17 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-17 16:00:00', 'YYYY-MM-DD HH24:MI:SS'), null),
(104, 4, TO_TIMESTAMP('2025-04-18 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-18 18:00:00', 'YYYY-MM-DD HH24:MI:SS'), null),
(105, 5, TO_TIMESTAMP('2025-04-19 16:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-19 20:00:00', 'YYYY-MM-DD HH24:MI:SS'), null);
```

```
INSERT INTO VEHICLE (VEHICLE_ID, VEHICLE_TYPE, MODEL, NAME, AGE, VIN) VALUES
(301, 1, 'Siemens Charger', 'Silver Meteor 1', TO_TIMESTAMP('2000-05-08 00:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'JNRAS08U06X162975'),
(302, 1, 'Siemens Charger', 'Silver Meteor 2', TO_TIMESTAMP('2018-05-19 00:00:00', 'YYYY-MM-DD HH24:MI:SS'), '1HSSDAAN8XH640604'),
(303, 1, 'Siemens Charger', 'Silver Meteor 3', TO_TIMESTAMP('2012-02-12 00:00:00', 'YYYY-MM-DD HH24:MI:SS'), '3GSCL33P48S670688'),
(304, 1, 'Siemens Charger', 'Silver Meteor 4', TO_TIMESTAMP('2020-04-05 00:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'JTEDS42A4920093855'),
(305, 0, 'Motor Coach', 'Sunshine Express', TO_TIMESTAMP('2015-04-13 00:00:00', 'YYYY-MM-DD HH24:MI:SS'), '2HGES26742H527362');
```

```
INSERT INTO STAFF (STAFF_ID, L_NAME, F_NAME, EMAIL, PHONE, ROLE, VEHICLE_ID, ASSIGNED_STATION_CODE) VALUES
(1, 'Tommie', 'Tran', NULL, 7306216796, 'Conductor', 301, 'TPA'),
(2, 'Dean', 'Lozano', NULL, 7302106471, 'Conductor', 302, 'ORL'),
(3, 'Buster', 'Lloyd', NULL, 9842228662, 'Clerk', null, 'MIA'),
(4, 'Michel', 'Schwartz', NULL, 7302247792, 'Conductor', 303, 'WPB'),
(5, 'Johnnie', 'Lutz', NULL, 7302444046, 'Conductor', 304, 'JAX'),
(6, 'Hans', 'Garrison', NULL, 6620353384, 'Driver', 305, 'MIA');
```

```
INSERT INTO ROUTE (ROUTE_ID, NAME, ORIGIN_ID, DEST_ID, VEHICLE_ID, VEHICLE_TYPE, DURATION_MINUTES, DISTANCE_MILES) VALUES
(401, 'Tampa-Orlando', 'TPA', 'ORL', 301, 1, 240, 84),
(402, 'Orlando-Miami', 'ORL', 'MIA', 302, 1, 210, 235),
(403, 'Miami-West Palm Beach', 'MIA', 'WPB', 303, 1, 90, 70),
(404, 'West Palm Beach-Jacksonville', 'WPB', 'JAX', 304, 1, 300, 271),
(405, 'Tampa-Jacksonville', 'TPA', 'JAX', 305, 0, 360, 200);
```

```sql
INSERT INTO SCHEDULE (SCHEDULE_ID, ROUTE_ID, DEPARTURE_TIME, ARRIVAL_TIME, DAYS_OF_WEEK) VALUES
(501, 401, TO_TIMESTAMP('2025-04-01 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(1,1,1,1,1,1,1)), -- Tampa to Orlando, every day
(502, 402, TO_TIMESTAMP('2025-04-01 12:30:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 16:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(1,1,1,1,1,1,1)), -- Orlando to Miami, every day
(503, 402, TO_TIMESTAMP('2025-04-01 10:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 13:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,1,1,1,1,1,0)), -- Orlando to Miami, every weekday
(504, 403, TO_TIMESTAMP('2025-04-01 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 15:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,1,1,1,1,1,0)), -- Miami to West Palm Beach, every weekday
(505, 404, TO_TIMESTAMP('2025-04-01 16:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 18:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,1,1,1,1,1,0)), -- West Palm Beach to Jacksonville, every weekday
(506, 403, TO_TIMESTAMP('2025-04-01 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 13:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(1,0,0,0,0,0,1)), -- Miami to West Palm Beach, every weekend
(507, 404, TO_TIMESTAMP('2025-04-01 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 19:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(1,0,0,0,0,0,1)), -- West Palm Beach to Jacksonville, every weekend
(508, 404, TO_TIMESTAMP('2025-04-01 14:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 19:30:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,1,1,1,0,0,0)), -- West Palm Beach to Jacksonville, every mon,tue,wen
(509, 405, TO_TIMESTAMP('2025-04-01 06:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,1,1,1,0,0,0)), -- Tampa to Jacksonville, every mon,tue,wen
(510, 405, TO_TIMESTAMP('2025-04-01 16:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 22:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,0,0,0,1,1,0)), -- Tampa to Jacksonville, every thur,fri
(511, 401, TO_TIMESTAMP('2025-04-01 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,0,0,0,1,1,0)), -- Tampa to Orlando, every thur,fri
(512, 402, TO_TIMESTAMP('2025-04-01 08:00:00', 'YYYY-MM-DD HH24:MI:SS'), TO_TIMESTAMP('2025-04-01 12:00:00', 'YYYY-MM-DD HH24:MI:SS'), GET_DAY_MASK(0,0,0,0,1,1,0)); -- Orlando to Miami, every thur,fri
```

```sql
INSERT INTO ROUTE_TICKET (ROUTE_TICKET_ID, ORDER_ID, TICKET_ID, SCHEDULE_ID) VALUES
-- Robert Waters trip: Tampa -> Orlando -> Miami
(601, 1, 101, 501), -- Tampa to Orlando
(602, 2, 101, 502), -- Orlando to Miami

-- Alan Benson trip: Orlando -> Miami -> West Palm Beach -> Jacksonville
(603, 1, 102, 503), -- Orlando to Miami
(604, 2, 102, 504), -- Miami to West Palm Beach
(605, 3, 102, 505), -- West Palm Beach to Jacksonville

-- Janet Friedman trip: Miami -> West Palm Beach -> Jacksonville
(606, 1, 103, 506), -- Miami to West Palm Beach
(607, 2, 103, 507), -- West Palm Beach to Jacksonville

-- Eunice Griffith trip: West Palm Beach -> Jacksonville -> Tampa
(608, 1, 104, 508), -- West Palm Beach to Jacksonville
(609, 2, 104, 509), -- Tampa to Jacksonville (return trip next day)

-- Matt Oneal trip: Tampa -> Jacksonville -> Tampa -> Orlando -> Miami
(610, 1, 105, 510), -- Tampa to Jacksonville
(611, 2, 105, 511), -- Tampa to Orlando (next day)
(612, 3, 105, 512); -- Orlando to Miami
```

SQL UPDATES

```sql
-- STATION table
UPDATE STATION
SET ZIP = '33605'
```

```sql
WHERE STATION_CODE = 'TPA';

UPDATE STATION
SET NAME = 'Orlando Regional Station', STREET = '1500 Sligh Blvd'
WHERE CITY = 'Orlando';
```

```sql
-- PASSENGER table
UPDATE PASSENGER
SET PHONE = '4342101934'
WHERE PASSENGER_ID = 1;

UPDATE PASSENGER
SET ADDRESS = 'Updated Address, Some City, Some State - 12345, USA'
WHERE EMAIL LIKE '%@outlook.com';
```

```sql
-- TICKET table
UPDATE TICKET
SET CONFIRMATION_CODE = 'ABC-123-DEF'
WHERE TICKET_ID = 101;

UPDATE TICKET
SET ARRIVAL_TIME = DEPARTURE_TIME + INTERVAL '4' HOUR
WHERE PASSENGER_ID IN (SELECT PASSENGER_ID FROM PASSENGER WHERE NAME LIKE 'Alan%');
```

```sql
-- STAFF table
UPDATE STAFF
SET ROLE = 'Driver', VEHICLE_ID = 305
WHERE STAFF_ID = 3;

UPDATE STAFF
SET ROLE = 'Clerk', VEHICLE_ID = null
WHERE STAFF_ID = 6;
```

```sql
-- VEHICLE table
UPDATE VEHICLE
SET NAME = 'Silver Comet 1'
WHERE VEHICLE_ID = 301;

UPDATE VEHICLE
SET AGE = AGE + INTERVAL '1' YEAR
WHERE VEHICLE_TYPE = 1 AND MODEL LIKE '%Charger%';
```

```sql
-- ROUTE table
UPDATE ROUTE
SET DURATION_MINUTES = 250
```

```sql
WHERE ROUTE_ID = 401;

UPDATE ROUTE
SET DISTANCE_MILES = DISTANCE_MILES + 10
WHERE ORIGIN_ID = 'ORL' AND DEST_ID = 'MIA';
```

```sql
-- SCHEDULE table
UPDATE SCHEDULE
SET DEPARTURE_TIME = TO_TIMESTAMP('2025-04-01 08:30:00', 'YYYY-MM-DD HH24:MI:SS')
WHERE SCHEDULE_ID = 501;

UPDATE SCHEDULE
SET ARRIVAL_TIME = DEPARTURE_TIME + INTERVAL '3' HOUR + INTERVAL '45' MINUTE
WHERE ROUTE_ID IN (SELECT ROUTE_ID FROM ROUTE WHERE NAME LIKE 'Miami%');
```

SQL QUERIES

```sql
--list all customers and number of tickets they have
SELECT
   p.NAME, p.EMAIL, p.PHONE, p.ADDRESS,
   COUNT(t.TICKET_ID) AS Amount_Of_Tickets
FROM PASSENGER p
JOIN TICKET t ON p.PASSENGER_ID = t.PASSENGER_ID
GROUP BY p.PASSENGER_ID, p.NAME, p.EMAIL, p.PHONE, p.ADDRESS
ORDER BY p.NAME;
```

NAME	EMAIL	PHONE	ADDRESS	AMOUNT_OF_TICKETS
Alan Benson	alanbenson21@outlook.com	2.73E+09	Updated Address, Some City, Some State - 12345, USA	1
Eunice Griffith	eunicegriffith78@hotmail.com	8.06E+09	79241 Doug Center, Lake Mafaldaworth, Arkansas - 83793	1
Janet Friedman	janetfriedman63@yahoo.com	6.63E+09	2834 Laurianne Loaf, Ortizberg, New Mexico - 42339, Eritrea	1
Matt Oneal	mattoneal46@protonmail.com	8.4E+09	463 Mervin Junctions, Port Keon, Colorado - 73593, Australia	1
Robert Waters	robertwalter91@gmail.com	4.34E+09	2002 Magdalena Pass, Pacochahaven, Tennessee 98096	1

```sql
--list all the schedules routes stations and staff from a ticket in the correct order 1 , 2, 3
SELECT
   rt.ORDER_ID as Orders,
   FORMAT_TIMESTAMP_DAY(s.DEPARTURE_TIME) AS DEPARTURE,
FORMAT_TIMESTAMP_DAY(s.ARRIVAL_TIME) as ARRIVAL,
   r.NAME AS RouteName,
   origin_station.NAME AS Origin_Station,
   dest_station.NAME AS Dest_Station,
```

```
  t.CONFIRMATION_CODE as Code
FROM TICKET t
JOIN ROUTE_TICKET rt ON t.TICKET_ID = rt.TICKET_ID
JOIN SCHEDULE s ON rt.SCHEDULE_ID = s.SCHEDULE_ID
JOIN ROUTE r ON s.ROUTE_ID = r.ROUTE_ID
JOIN STATION origin_station ON r.ORIGIN_ID = origin_station.STATION_CODE
JOIN STATION dest_station ON r.DEST_ID = dest_station.STATION_CODE
  WHERE t.TICKET_ID = 102
  ORDER BY rt.ORDER_ID;
```

ORDERS	DEPARTURE	ARRIVAL	ROUTENAME	ORIGIN_STATION	DEST_STATION	CODE
1	April 01 2025, 10:00 AM	April 01 2025, 01:30 PM	Orlando-Miami	Orlando Regional Station	Miami Amtrak Station	BC8DHN799B
2	April 01 2025, 02:00 PM	April 01 2025, 03:30 PM	Miami-West Palm Beach	Miami Amtrak Station	West Palm Beach Amtrak Station	BC8DHN799B
3	April 01 2025, 04:00 PM	April 01 2025, 06:30 PM	West Palm Beach-Jacksonville	West Palm Beach Amtrak Station	Jacksonville Amtrak Station	BC8DHN799B

```
--List all staff and their vehicles and assigned station
SELECT
  (s.F_NAME || ' ' || s.L_NAME) as Staff_Name, s.ROLE,
  v.NAME AS Vehicle_Name, v.MODEL, VEHICLE_TYPE_CHAR(v.VEHICLE_TYPE) as Vehicle, v.VIN,
  n.STATION_CODE, n.NAME as STATION_NAME,
  (n.STREET || ' ' || n.CITY || ' ' || n.STATE || ' ' || n.ZIP || ' ' || n.LATITUDE || ' ' || n.LONGITUDE) as
STATION_ADDRESS
FROM STAFF s
JOIN STATION n on s.ASSIGNED_STATION_CODE = n.STATION_CODE
LEFT JOIN VEHICLE v ON s.VEHICLE_ID = v.VEHICLE_ID
ORDER BY s.L_NAME, s.F_NAME;
```

STAFF_NAME	ROLE	VEHICLE_NAME	MODEL	VEHICLE	VIN	STATION_CODE	STATION_NAME	STATION_ADDRESS
Lloyd Buster	Driver	Sunshine Express	Motor Coach	BUS	2HGES26742H527362	MIA	Miami Amtrak Station	8303 NW 37th Ave Miami FL 33147 25.8115 -80.2227
Lozano Dean	Driver	Silver Meteor 2	Siemens Charger	TRAIN	1HSSDAAN8XH640604	ORL	Orlando Regional Station	1500 Sligh Blvd Orlando FL 32806 28.5199 -81.382
Lutz Johnnie	Conductor	Silver Meteor 4	Siemens Charger	TRAIN	JTEDS42A492009385	JAX	Jacksonville Amtrak Station	3570 Clifford Ln Jacksonville FL 32209 30.3392 -81.6543
Schwartz Michel	Driver	Silver Meteor 3	Siemens Charger	TRAIN	3GSCL33P48S670688	WPB	West Palm Beach Amtrak Station	203 S Tamarind Ave West Palm Beach FL 33401 26.7118 -80.0526
Tran Tommie	Conductor	Silver Comet 1	Siemens Charge	TRAIN	JNRAS08U06X162975	TPA	Tampa Union Station	601 N Nebraska Ave Tampa FL 33605 27.9515 -82.4552

r

```
--List a station's routes, and schedules
SELECT
   r.NAME AS RouteName, VEHICLE_TYPE_CHAR(r.VEHICLE_TYPE) as Vehicle, r.DURATION_MINUTES,
r.DISTANCE_MILES,
   FORMAT_TIMESTAMP(s.DEPARTURE_TIME) ||  GET_DAY_STRING(s.DAYS_OF_WEEK) AS DEPARTURE,
FORMAT_TIMESTAMP(s.ARRIVAL_TIME) as ARRIVAL
FROM STATION st
JOIN ROUTE r ON st.STATION_CODE = r.ORIGIN_ID OR st.STATION_CODE = r.DEST_ID
JOIN SCHEDULE s ON r.ROUTE_ID = s.ROUTE_ID
WHERE st.STATION_CODE = 'JAX'
ORDER BY r.ROUTE_ID, s.SCHEDULE_ID;
```

ROUTENAME	VEHICLE	DURATION_MINUTES	DISTANCE_MILES	DEPARTURE	ARRIVAL
West Palm Beach-Jacksonville	TRAIN	300	271	April 2025, 04:00 PM Every Weekday	April 2025, 06:30 PM
West Palm Beach-Jacksonville	TRAIN	300	271	April 2025, 02:00 PM Every Weekend	April 2025, 07:30 PM
West Palm Beach-Jacksonville	TRAIN	300	271	April 2025, 02:00 PM Every Mon, Tue, Wed	April 2025, 07:30 PM
Tampa-Jacksonville	BUS	360	200	April 2025, 06:00 AM Every Mon, Tue, Wed	April 2025, 08:00 AM
Tampa-Jacksonville	BUS	360	200	April 2025, 04:00 PM Every Thu, Fri	April 2025, 10:00 PM

```
--List all schedules with their source and destination stations
SELECT
   FORMAT_TIMESTAMP(s.DEPARTURE_TIME) ||  GET_DAY_STRING(s.DAYS_OF_WEEK) AS DEPARTURE,
FORMAT_TIMESTAMP(s.ARRIVAL_TIME) as ARRIVAL,
   r.DURATION_MINUTES as TYPICAL_DURATION_MINUTES, EXTRACT(HOUR FROM (s.ARRIVAL_TIME -
s.DEPARTURE_TIME)) * 60 + EXTRACT(MINUTE FROM(s.ARRIVAL_TIME - s.DEPARTURE_TIME)) as
ACTUAL_DURATION, r.DISTANCE_MILES,
   origin_station.STATION_CODE AS Origin_Name,
   destination_station.STATION_CODE AS Dest_Name
FROM SCHEDULE s
JOIN ROUTE r ON s.ROUTE_ID = r.ROUTE_ID
JOIN STATION origin_station ON r.ORIGIN_ID = origin_station.STATION_CODE
JOIN STATION destination_station ON r.DEST_ID = destination_station.STATION_CODE
ORDER BY s.SCHEDULE_ID;
```

DEPARTURE	ARRIVAL	TYPICAL_DURATION_MINUTES	ACTUAL_DURATION	DISTANCE_MILES	ORIGIN_NAME	DEST_NAME
April 2025, 08:00 AM Every Day	April 2025, 12:00 PM	240	240	84	TPA	ORL
April 2025, 12:30 PM Every Day	April 2025, 04:00	210	210	235	ORL	MIA

	PM					
April 2025, 10:00 AM Every Weekday	April 2025, 01:30 PM	210	210	235	ORL	MIA
April 2025, 02:00 PM Every Weekday	April 2025, 03:30 PM	90	90	70	MIA	WPB
April 2025, 04:00 PM Every Weekday	April 2025, 06:30 PM	300	150	271	WPB	JAX
April 2025, 12:00 PM Every Weekend	April 2025, 01:30 PM	90	90	70	MIA	WPB
April 2025, 02:00 PM Every Weekend	April 2025, 07:30 PM	300	330	271	WPB	JAX
April 2025, 02:00 PM Every Mon, Tue, Wed	April 2025, 07:30 PM	300	330	271	WPB	JAX
April 2025, 06:00 AM Every Mon, Tue, Wed	April 2025, 08:00 AM	360	120	200	TPA	JAX
April 2025, 04:00 PM Every Thu, Fri	April 2025, 10:00 PM	360	360	200	TPA	JAX
April 2025, 08:00 AM Every Thu, Fri	April 2025, 12:00 PM	240	240	84	TPA	ORL
April 2025, 08:00 AM Every Thu, Fri	April 2025, 12:00 PM	210	240	235	ORL	MIA

SQL Constraints

```
-- CHECK constraint violation (via trigger): Invalid ROLE
INSERT INTO STAFF (STAFF_ID, L_NAME, F_NAME, EMAIL, PHONE, ROLE)
VALUES (203, 'Williams', 'Charlie', null, '5555556666', 'TEACHER');
error number argument to raise_application_error of -10005 is out of range
```

```
-- NOT NULL constraint violation: Not Nullable ARRIVAL_TIME/DEPARTURE_TIME
INSERT INTO TICKET (TICKET_ID, PASSENGER_ID, DEPARTURE_TIME, CONFIRMATION_CODE)
VALUES (104, 2, null, null);
cannot insert NULL into ("SQL_PNKFOYYUY27OU9V24KQDVKSCL9"."TICKET"."DEPARTURE_TIME")
```

```
-- PRIMARY KEY constraint violation: Duplicate STAFF_ID
INSERT INTO STAFF (STAFF_ID, L_NAME, F_NAME, EMAIL, PHONE)
VALUES (2, 'Another Staff', 'Person', NULL, '1119992222')
error number argument to raise_application_error of -10004 is out of range
```

```
-- UNIQUE constraint violation: Duplicate VIN
INSERT INTO VEHICLE (VEHICLE_ID, VEHICLE_TYPE, MODEL, NAME, AGE, VIN)
```

```
VALUES (304, 1, 'Superliner', 'Long Haul Train', TIMESTAMP '2030-03-01 00:00:00', '2HGES26742H527362');
ORA-00001: unique constraint (SQL_PNKFOYYUY27OU9V24KQDVKSCL9.SYS_C001154910) violated on table
SQL_PNKFOYYUY27OU9V24KQDVKSCL9.VEHICLE columns (VEHICLE_ID)
```

References

Personal Experience: Rode a couple of trips throughout Florida and used their mobile/web app.

What is Amtrak:

- https://en.wikipedia.org/wiki/Amtrak

Borrowed Presentation Slides:

- https://media.amtrak.com/wp-content/uploads/2018/02/Amtrak-Dec.-2017-PTC-Deck.pdf

Data/Rules:

- https://catalog.data.gov/dataset/amtrak-routes1
- https://geodata.bts.gov/datasets/usdot::amtrak-routes/about
- https://geodata.bts.gov/datasets/usdot::amtrak-stations/explore
- https://github.com/piemadd/amtrak